

Module 9.1 - Template-Driven Form Basics

Learning Objectives:

- Understand what template-driven forms are
 - Set up FormsModule in your application
 - Use ngModel for two-way data binding
 - Access form values and states with ngForm
 - Submit and handle form data
-

Table of Contents

1. [What is Template-Driven Form?](#)
 2. [Setting Up Forms](#)
 3. [ngModel Directive](#)
 4. [ngForm Directive](#)
 5. [Form Submission](#)
 6. [Summary](#)
-

1. What is Template-Driven Form?

1.1 Overview

Template-Driven Forms are forms where the form logic is defined in the HTML template using directives. Angular handles the form object creation automatically.

Key Characteristics:

- Logic is in the template (HTML)
- Uses directives like ngModel

- Angular creates FormControl automatically
- Best for simple forms

1.2 When to Use

Use Template-Driven	Use Reactive (Module 10)
Simple forms	Complex forms
Less validation logic	Dynamic validation
Quick prototypes	Testing required
Logic in template	Logic in component

2. Setting Up Forms

2.1 Import FormsModule

Step 1: Import FormsModule in your module:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms'; // Import this

import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    FormsModule // Add here
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

2.2 Basic Form Structure

Simple form template:

```
<form>
  <input type="text" name="username">
  <button type="submit">Submit</button>
</form>
```

☑ That's it! Angular automatically creates a form object for any `<form>` tag.

3. ngModel Directive

3.1 What is ngModel?

`ngModel` creates two-way data binding between the template and component.

Basic Syntax:

```
<input [(ngModel)]="propertyName" name="fieldName">
```

⚠ **Important:** You must include the `name` attribute when using `ngModel` in a form!

3.2 Simple Example

Component:

```
export class AppComponent {
  username = '';
}
```

Template:

```
<form>
  <input [(ngModel)]="username" name="username">
  <p>You typed: {{ username }}</p>
</form>
```

☒ Whatever you type in the input instantly appears in the paragraph!

3.3 Multiple Fields

Component:

```
export class AppComponent {
  user = {
    name: '',
    email: ''
  };
}
```

Template:

```
<form>
  <input [(ngModel)]="user.name" name="name">
  <input [(ngModel)]="user.email" name="email">

  <p>Name: {{ user.name }}</p>
  <p>Email: {{ user.email }}</p>
</form>
```

4. ngForm Directive

4.1 What is ngForm?

ngForm is automatically applied to every `<form>` tag. It tracks:

- Form values

- Form state (valid, touched, etc.)
- Individual field states

4.2 Accessing the Form

Template Reference Variable:

```
<form #myForm="ngForm">
  <input [(ngModel)]="username" name="username">
</form>

<pre>{{ myForm.value | json }}</pre>
```

☒ `myForm.value` gives you an object with all form values!

4.3 Form Properties

Useful properties:

```
<form #f="ngForm">
  <input [(ngModel)]="email" name="email">
</form>

<p>Form Value: {{ f.value | json }}</p>
<p>Valid: {{ f.valid }}</p>
<p>Touched: {{ f.touched }}</p>
<p>Dirty: {{ f.dirty }}</p>
```

Property	Description
<code>value</code>	Object with all form values
<code>valid</code>	True if form passes validation
<code>touched</code>	True if user interacted with form
<code>dirty</code>	True if user changed any value
<code>pristine</code>	True if form unchanged (opposite of dirty)

4.4 Field-Level Access

Access individual field state:

```
<form #f="ngForm">
  <input
    [(ngModel)]="email"
    name="email"
    #emailField="ngModel">

  <p>Email Valid: {{ emailField.valid }}</p>
  <p>Email Touched: {{ emailField.touched }}</p>
</form>
```

5. Form Submission

5.1 Basic Submit

Template:

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input [(ngModel)]="username" name="username">
  <button type="submit">Submit</button>
</form>
```

Component:

```
export class AppComponent {
  username = '';

  onSubmit(form: any): void {
    console.log('Form submitted!');
    console.log('Values:', form.value);
  }
}
```

5.2 Complete Example

Component:

```
export class LoginComponent {
  credentials = {
    email: '',
    password: ''
  };

  onLogin(form: any): void {
    console.log('Login attempt:', form.value);
    // Here you would call a service to authenticate
  }
}
```

Template:

```
<form #loginForm="ngForm" (ngSubmit)="onLogin(loginForm)">
  <div>
    <label>Email</label>
    <input
      type="email"
      [(ngModel)]="credentials.email"
      name="email">
    </div>

    <div>
      <label>Password</label>
      <input
        type="password"
        [(ngModel)]="credentials.password"
        name="password">
      </div>

    <button type="submit">Login</button>
  </form>

  <!-- Debug info -->
  <pre>{{ loginForm.value | json }}</pre>
```

5.3 Disabling Submit Button

Disable button when form is invalid:

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input [(ngModel)]="email" name="email" required>

  <button
    type="submit"
    [disabled]="!f.valid">
    Submit
  </button>
</form>
```

6. Summary

6.1 Key Concepts

1. FormsModule

- Must import in module to use forms
- Provides ngModel and ngForm directives

2. ngModel

- Two-way data binding: `[(ngModel)]="property"`
- Must include `name` attribute
- Automatically creates FormControl

3. ngForm

- Automatically applied to `<form>` tags
- Access with template reference: `#f="ngForm"`
- Provides form value and state

4. Form Submission

- Use `(ngSubmit)` event
- Pass form reference to handler
- Access values via `form.value`

6.2 Quick Reference

Minimal Form:

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input [(ngModel)]="name" name="name">
  <button type="submit">Submit</button>
</form>
```

Component:

```
export class MyComponent {
  name = '';

  onSubmit(form: any): void {
    console.log(form.value);
  }
}
```

6.3 Form States Reference

State	True When
<code>valid</code>	All validations pass
<code>invalid</code>	Any validation fails
<code>pristine</code>	User hasn't changed value
<code>dirty</code>	User changed value
<code>touched</code>	User focused then blurred field

State	True When
untouched	User hasn't focused field

6.4 Common Patterns

Pattern 1: Display form values

```
<pre>{{ myForm.value | json }}</pre>
```

Pattern 2: Disable submit if invalid

```
<button [disabled]="!f.valid">Submit</button>
```

Pattern 3: Check field state

```
<input #field="ngModel" [(ngModel)]="value" name="field">
<div *ngIf="field.touched && field.invalid">Error!</div>
```

6.5 Next Steps

In the next module, you'll learn:

- Form validation (required, minlength, pattern)
- HTML5 validation vs Angular validation
- Custom validators
- Error messages display
- Form grouping with ngModelGroup

Additional Resources

- Forms Guide: <https://angular.io/guide/forms-overview>

- **Template-Driven Forms:** <https://angular.io/guide/forms>
- **FormsModule API:** <https://angular.io/api/forms/FormsModule>
- **ngModel:** <https://angular.io/api/forms/NgModel>

💡 **Pro Tip:** Template-driven forms are perfect for simple forms with basic validation. For complex forms with dynamic validation, use Reactive Forms (Module 10)!

